

Reviewer Pack — Minimal Coxeter Group Order in Representation Theory vs Perm...

Entry 78491bd9a2d9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 07:26:01 UTC

Minimal Coxeter Group Order in Representation Theory vs Permutation Circuit Complexity

Entry ID: 78491bd9a2d9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 07:26:01 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory: Coxeter Groups **Field B** (complexity object): Complexity Theory: Permutation Circuit Complexity

Statement:

[‘For any permutation f of n elements, there exists a Coxeter group C such that the minimal order of an element in C that generates the set of permutations generated by f is $\Omega(n^{2/4})$.’, ‘Moreover, for any permutation circuit C with depth d and width w , there exists a Coxeter group G such that the minimal order of an element in G generating the set of permutations represented by C is $\Omega((d + w)^2)$.’]

Rationale (proposer’s reasoning):

[‘The structure of Coxeter groups has been studied extensively in representation theory, revealing non-trivial algebraic properties. These properties could provide a new lens through which to analyze permutation circuits, potentially leading to lower bounds on their complexity.’, ‘Permutation circuits are a fundamental object in computational complexity theory, and finding connections to well-studied mathematical structures may expose deep insights into their behavior.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 76f0de6a169ee016

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, compute the minimal order of an element in a Coxeter group for given permutations or circuits. If the mean minimal order is $\Omega(n^{2/4})$ for permutations and $\Omega((d + w)^2)$ for circuits with a high degree of confidence (e.g., 95% CI), the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k = len(A), len(B[0])
    n = len(B)
    C = [[0 for _ in range(k)] for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(n):
                C[i][j] += A[i][l] * B[l][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(b)
    Augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
                max_row = j
        Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
        pivot = Augmented[i][i]
        for j in range(i, n+1):
            Augmented[i][j] /= pivot
        for j in range(i+1, m):
            for k in range(i, n+1):
                Augmented[j][k] -= Augmented[i][k] * Augmented[j][i]
```

```

        Augmented[i][j] /= pivot
    for j in range(m):
        if j != i:
            factor = Augmented[j][i]
            for k in range(n+1):
                Augmented[j][k] -= factor * Augmented[i][k]
    return [row[-1] for row in Augmented]

def calculate_circuit_group_order(circuit):
    d, w = len(circuit), len(circuit[0])
    if all(all(circuit[layer][i] == j % w for i in range(w)) for layer in range(d)):
        return 1
    G = [[0]*w for _ in range(w)]
    for layer in range(d):
        for i in range(w):
            for j in range(w):
                if circuit[layer][i] == (j + 1) % w:
                    G[i][j] = 1
    I = [[int(i==j) for j in range(w)] for i in range(w)]
    order = 1
    while True:
        G = matrix_multiply(G, G)
        order += 1
        if gaussian_elimination(G + I, [0]*w + [1]*w) == [1]*w:
            return order

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_order = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Ensure at least 5 instances per size
            perm = random.sample(range(n), n)
            circuit = [[random.randint(0, w-1) for _ in range(w)] for _ in range(d)]

            permutation_order = calculate_circuit_group_order([perm])
            total_order += permutation_order
            instances_tested += 1

            if permutation_order < n**2 / 4:
                return {
                    "metric_name": "permutation_order",
                    "metric_value": permutation_order,
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
                    "counterexample": f"Permutation {perm} has order {permutation_order}"
                }

    mean_order = total_order / instances_tested
    return {
        "metric_name": "permutation_order",
        "metric_value": mean_order,
        "instances_tested": instances_tested,
        "conjecture_holds": mean_order >= n**2 / 4,
    }

```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_order = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Permutation order is less than n^2/4\" first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ee062669.py", line 112, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ee062669.py", line 82, in run_trial
    circuit = [[random.randint(0, w-1) for _ in range(w)] for _ in range(d)]
                                                    ^
NameError: name 'd' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'd', which prevented it from producing any data. As a result, the pre-registered support condition could not be unambiguously

met. | next: Investigate and fix the error in the test code to ensure that it can run successfully and produce the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15764
2	propose	ollama_remote	glm4:latest	0	0	13650
3	preregistration	ollama_remote	glm4:latest	0	0	9703
4	novelty	ollama_remote	glm4:latest	0	0	8736
5	novelty	ollama_remote	glm4:latest	0	0	8569
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19175
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18505
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11017
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13211
10	judge	ollama_remote	glm4:latest	0	0	11742

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 130072 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/78491bd9a2d9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/78491bd9a2d9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/78491bd9a2d9.tar.gz (if generated)